

Филитов Миша

АКОС 3

динамическая память

НИУ ВШЭ ФКН ПМИ 2026

Storage Duration

Storage Duration

- ❖ **Automatic** - язык определяет момент создания и удаления
- ❖ **Static** - существует все время работы программы
- ❖ **Thread** - существует все время жизни потока
- ❖ **Allocated** - время жизни явно управляется программой

```
4 // static storage duration
5 int global = 10;
6
7 void f() {
8     // automatic storage duration
9     int automatic = 0;
10    // static storage duration
11    static int persistent = 0;
12
13    // p — automatic
14    // *p — allocated
15    int *p = malloc(sizeof(*p));
16
17    // освободили *p
18    // p еще существует
19    free(p);
20 }
```

Linkage

Linkage

- ❖ **No linkage** - имя относится только к этой области видимости
- ❖ **Internal linkage** - имя может обозначать тот же объект / функцию внутри одного translation unit
- ❖ **External linkage** - имя может обозначать тот же объект / функции в разных translation units

Linkage

```
6 // a.c
7
8 // external linkage
9 extern int shared = 10;
10 // internal linkage
11 static int hidden = 20;
12
13 // external linkage
14 extern void print_a() {
15     // no linkage
16     int i = 55;
17     printf("shared = %d\n", shared);
18     printf("hidden = %d\n", hidden);
19     printf("i = %d\n", i);
20 }
```

```
6 // b.c
7
8 // refers to the same object from a.c
9 extern int shared;
10
11 // same function declared in a.c
12 extern void print_a();
13
14 extern int main() {
15     printf("shared = %d\n", shared);
16     // linker error
17     // printf("hidden = %d\n", hidden);
18
19     shared = 42;
20     print_a();
21 }
```

Storage-class specifiers

Спецификаторы класса хранения

- ❖ **auto** - automatic duration, no linkage
- ❖ **register** - automatic duration, no linkage, address of this variable cannot be taken
- ❖ **static** - static duration and internal linkage (unless at block scope)
- ❖ **extern** - static duration and external linkage (unless already declared internal)
- ❖ **thread_local** - thread storage duration

```
3 // Static storage duration
4 // internal linkage
5 static int file_static = 10;
6
7 // Static storage duration
8 // external linkage
9 int file_extern = 20;
10
11 void g() {
12     // Static storage duration,
13     // no linkage
14     static int s = 3;
15     printf("s = %d\n", s++);
16 }
17
18 void f() {
19     // Automatic storage duration
20     // no linkage
21     auto int a = 1;
22
23     printf("a = %d\n", a);
24     printf("file_extern = %d\n", file_extern);
25     for (int i = 0; i < 3; ++i)
26         g();
27 }
```

Вот бы нам объекты, временем жизни
которых мы можем управлять

Проблема

- ❖ Для статических массивов нужно знать размеры в момент запуска программы
- ❖ VLA - variable-length array

```
3 // Надо знать размер в момент компиляции
4 int array[100];
5
6 √ int* get_array(int n) {
7     int arr[n];
8     // УБ
9     return arr;
10 }
```

<https://godbolt.org/z/Pfz5rj1oz>

Динамическая память

Как работать с динамической памятью

❖ malloc(size)

- ❖ Выделяет size байт
- ❖ Содержимое не инициализируется

❖ calloc(count, size)

- ❖ Выделяет память под count элементов по size байт
- ❖ Память обнуляется

❖ realloc(ptr, new_size)

- ❖ Изменяет размер ранее выделенного блока
- ❖ Может вернуть другой адрес

❖ free(ptr)

- ❖ Освобождает ранее выделенную память

❖ reallocarray(ptr, count, size)

- ❖ Аналог realloc(ptr, count * size)
- ❖ но с проверкой переполнения count * size

```
1  #include <stdlib.h>
2
3  void *malloc(size_t size);
4  void free(void *ptr);
5
6  void *calloc(size_t count, size_t size);
7  void *realloc(void *ptr, size_t new_size);
8
9  // non-standard
10 void *reallocarray(void *ptr, size_t count, size_t size);
11
12 int main() {
13     int* p = malloc(sizeof(*p));
14     free(p);
15 }
```

<https://godbolt.org/z/habPGeEbf>

```

4  int main() {
5      //    automatic storage duration
6      //    |
7      //    v
8      int *p = malloc(sizeof(*p));
9      //      ^
10     //      |
11     //      + -- allocated storage duration
12
13     *p = 42;
14
15     printf("&p = %p\n", (void *)&p);
16     printf(" p = %p\n", (void *)p);
17     printf("*p = %d\n", *p);
18
19     free(p);
20
21     return 0;

```

<https://godbolt.org/z/zTGrTWeao>

```

4  int *foo() {
5      int *ptr = malloc(sizeof(*ptr));
6
7      printf("foo:\n");
8      printf("  &ptr = %p\n", (void *)&ptr);
9      printf("  ptr = %p\n", (void *)ptr);
10
11     return ptr;
12 }
13
14 int main() {
15     int *ptr = foo();
16
17     printf("main:\n");
18     printf("  &ptr = %p\n", (void *)&ptr);
19     printf("  ptr = %p\n", (void *)ptr);
20
21     free(ptr);
22     return 0;
23 }

```

Program returned: 0

Program stdout

foo:

&ptr = 0x7ffeba9abd98

ptr = 0x388432b0

main:

&ptr = 0x7ffeba9abdb8

ptr = 0x388432b0

<https://godbolt.org/z/ef146W6nq>

malloc()

- ❖ Память не инициализирована
- ❖ Не важно, массив или объект
- ❖ Писать/читать можно только в рамках выделенного места
- ❖ Надо проверять указатель на NULL

```
4  int main() {
5      // память под 1 инт (4 байта)
6      int *x = malloc(sizeof(*x));
7
8      // Память по 10 интов (40 байт)
9      int *arr = malloc(10 * sizeof(*arr));
10
11     // .....
12
13     // free одинаковый
14     free(x);
15     free(arr);
16
17     return 0;
18 }
```

x86-64 gcc 16.2



-fsanitize=address

Program returned: 1

Program stderr

```
=====
==2==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x7adfd87e0044 at pc 0x0000000000000000
WRITE of size 4 at 0x7adfd87e0044 thread T0
```

#0 0x0000000000000000 in main /app/example.c:29

#1 0x7e9fd9a2a1c9 (/lib/x86_64-linux-gnu/libc.so.6+0x2a1c9) (BuildId: a4a7992a8e66!)

#2 0x7e9fd9a2a28a in __libc_start_main (/lib/x86_64-linux-gnu/libc.so.6+0x2a28a) (Bu

#3 0x00000000000000d4 in _start (/app/output.s+0x4010d4) (BuildId: 7e0291563dd0fe17a978)

0x7adfd87e0044 is located 4 bytes after 48-byte region [0x7adfd87e0010,0x7adfd87e0040)

allocated by thread T0 here:

#0 0x7e9fd9df11df in malloc (/opt/compiler-explorer/gcc-16.2.0/lib64/libasan.so.8+0x

#1 0x0000000000000000 in main /app/example.c:20

#2 0x7e9fd9a2a1c9 (/lib/x86_64-linux-gnu/libc.so.6+0x2a1c9) (BuildId: a4a7992a8e66!)

#3 0x7e9fd9a2a28a in __libc_start_main (/lib/x86_64-linux-gnu/libc.so.6+0x2a28a) (Bu

#4 0x00000000000000d4 in _start (/app/output.s+0x4010d4) (BuildId: 7e0291563dd0fe17a978)

SUMMARY: AddressSanitizer: heap-buffer-overflow /app/example.c:29 in main

Shadow bytes around the buggy address:

0x7adfd87dfd80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

0x7adfd87dfe00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

0x7adfd87dfe80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

0x7adfd87dff00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

0x7adfd87dff80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

=>0x7adfd87e0000: fa fa 00 00 00 00 00 00 [fa] fa fa fa fa fa fa fa

0x7adfd87e0080: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa

0x7adfd87e0100: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa

0x7adfd87e0180: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa

0x7adfd87e0200: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa

0x7adfd87e0280: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa

Shadow byte legend (one shadow byte represents 8 application bytes):

Addressable: 00

Partially addressable: 01 02 03 04 05 06 07

Heap left redzone: fa

Freed heap region: fd

Stack left redzone: f1

Stack mid redzone: f2

Stack right redzone: f3

Stack after return: f5

Stack use after scope: f8

Global redzone: f9

Global init order: f6

Poisoned by user: f7

Container overflow: fc

Array cookie: ac

Intra object redzone: bb

ASan internal: fe

Left alloca redzone: ca

Right alloca redzone: cb

==2==ABORTING

```

5  typedef struct {
6      char c1;
7      char c2;
8      int i;
9      double d1;
10     double d2;
11 } MyStruct;
12
13 int main() {
14     size_t size = SIZE_MAX / 8 + 3;
15
16     printf("elements wanted = %zu\n", size);
17     printf("bytes calculated = %zu\n",
18           size * sizeof(MyStruct));
19
20     MyStruct *ptr = malloc(size * sizeof(*ptr));
21     if (ptr == NULL)
22         return 1;
23
24     // malloc received 48 bytes:
25     // enough for only two MyStruct objects.
26     ptr[0].i = 10; // OK
27     ptr[1].i = 20; // OK
28
29     ptr[2].i = 30; // UB: out of bounds
30
31     free(ptr);
32 }

```

calloc()

- ❖ Принимает количество и размер
- ❖ Сам вернет NULL при переполнении
- ❖ Память заполнена нулями
- ❖ Это часто может быть быстрее, чем malloc+memset

```
5  int main() {
6      size_t n = 10;
7
8      // calloc: allocate + zero-initialize
9      int *a = calloc(n, sizeof(*a));
10
11     // malloc + memset
12     int *b = malloc(n * sizeof(*b));
13     if (b != NULL) {
14         memset(b, 0, n * sizeof(*b));
15     }
16     // check a != NULL && b != NULL
17     for (size_t i = 0; i < n; ++i) {
18         printf("%d %d\n", a[i], b[i]);
19     }
20
21     free(a);
22     free(b);
23
24     return 0;
25 }
```

realloc()

- ❖ Увеличить / уменьшить ранее выделенный блок
- ❖ Блок может остаться на том же адресе, а может переехать;
- ❖ При неудаче старый блок не освобождается
- ❖ Надо не потерять указатель

```
3  int main() {
4      int *arr = malloc(10 * sizeof(*arr));
5
6      // плохо! Если realloc вернет null, то потеряли
7      // указатель
8      // arr = realloc(arr, 20 * sizeof(*arr));
9      int *tmp = realloc(arr, 20 * sizeof(*arr));
10
11     if (tmp == NULL) {
12         // arr is still valid
13         free(arr);
14         return 1;
15     }
16
17     arr = tmp;
18
19     free(arr);
20     return 0;
21 }
```

free()

- ❖ Можно вызывать только для указателя, полученного из аллокатора
- ❖ Можно передавать только адрес, полученный от аллокатора
- ❖ Один и тот же блок нельзя освободить дважды
- ❖ Нельзя читать / писать освобожденную память
- ❖ free(NULL) корректно
- ❖ Нет способа проверить, освобождена ли память по указателю

```
4 void f1() {
5     int *p = malloc(10 * sizeof(*p));
6
7     free(p); // ОК
8     free(p); // нельзя
9 }
10
11 void f2() {
12     int *p = malloc(10 * sizeof(*p));
13
14     free(p); // ОК
15     p = NULL;
16     free(p); // ОК
17 }
18
19 void f() {
20     int *p = malloc(10 * sizeof(*p));
21     free(p + 1); // нельзя
22 }
```

Как ловить проблемы с памятью?

Все ли тут в порядке с кодом?

```
5 char* copy_string(const char *src) {
6     size_t len = strlen(src);
7
8     char *dst = malloc(len);
9     if (dst == NULL)
10         return NULL;
11
12     strcpy(dst, src);
13
14     return dst;
15 }
16
17 int main() {
18     char *s = copy_string("hello");
19
20     printf("%s\n", s);
21
22     free(s);
23     return 0;
24 }
```

x86-64 gcc 16.2

Program returned: 0

Program stdout

hello

Address sanitizer

```
5 char* copy_string(const char *src) {
6     size_t len = strlen(src);
7
8     // BUG: len+1
9     char *dst = malloc(len);
10    if (dst == NULL)
11        return NULL;
12
13    strcpy(dst, src);
14
15    return dst;
16 }
17
18 int main() {
19     char *s = copy_string("hello");
20
21     printf("%s\n", s);
22
23     free(s);
24     return 0;
25 }
26
```

x86-64 gcc 16.2

-fsanitize=address

Program returned: 1

Program stderr

=====
==2==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x6f59755e0015 at pc 0x733976c6c562 bp 0x7ffe6d464c30
WRITE of size 6 at 0x6f59755e0015 thread T0

#0 0x733976c6c561 (/opt/compiler-explorer/gcc-16.2.0/lib64/libasan.so.8+0xc0561) (BuildId: 792f44c80230c40040b1)
#1 0x0000004011f2 in copy_string /app/example.c:13
#2 0x00000040120a in main /app/example.c:19
#3 0x73397682a1c9 (/lib/x86_64-linux-gnu/libc.so.6+0x2a1c9) (BuildId: a4a7992a8e66555c8141ab2a08a8465ff6e0ea65)
#4 0x73397682a28a in __libc_start_main (/lib/x86_64-linux-gnu/libc.so.6+0x2a28a) (BuildId: a4a7992a8e66555c8141ab2a08a8465ff6e0ea65)
#5 0x0000004010e4 in _start (/app/output.s+0x4010e4) (BuildId: 836dfca3acd548bd22227db925873efc002d5a48)

0x6f59755e0015 is located 0 bytes after 5-byte region [0x6f59755e0010,0x6f59755e0015)
allocated by thread T0 here:

#0 0x733976cd81df in malloc (/opt/compiler-explorer/gcc-16.2.0/lib64/libasan.so.8+0x12c1df) (BuildId: 792f44c80230c40040b1)
#1 0x0000004011cd in copy_string /app/example.c:9
#2 0x00000040120a in main /app/example.c:19
#3 0x73397682a1c9 (/lib/x86_64-linux-gnu/libc.so.6+0x2a1c9) (BuildId: a4a7992a8e66555c8141ab2a08a8465ff6e0ea65)
#4 0x73397682a28a in __libc_start_main (/lib/x86_64-linux-gnu/libc.so.6+0x2a28a) (BuildId: a4a7992a8e66555c8141ab2a08a8465ff6e0ea65)
#5 0x0000004010e4 in _start (/app/output.s+0x4010e4) (BuildId: 836dfca3acd548bd22227db925873efc002d5a48)

SUMMARY: AddressSanitizer: heap-buffer-overflow /app/example.c:13 in copy_string

Shadow bytes around the buggy address:

0x6f59755dfd80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x6f59755dfe00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x6f59755dfe80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x6f59755dff00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x6f59755dff80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
=>0x6f59755e0000: fa fa[05]fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x6f59755e0080: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x6f59755e0100: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x6f59755e0180: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x6f59755e0200: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x6f59755e0280: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa

<https://godbolt.org/z/n9r3vhd1j>

Санитайзеры

- ❖ Address - use-after-free, use-after-return, double-free
- ❖ Thread - datarace
- ❖ Memory - чтение неинициализированной памяти, неинициализированный указатель
- ❖ Undefined - переполнение int

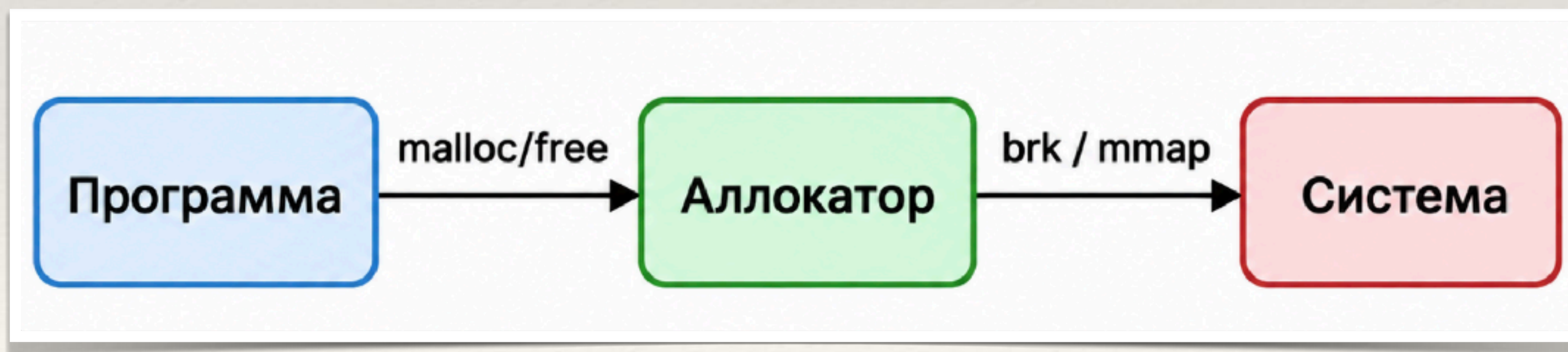
Valgrind

- ❖ Не требует пересборки
- ❖ Время работы сильно выше
- ❖ Инструментирует во время выполнения

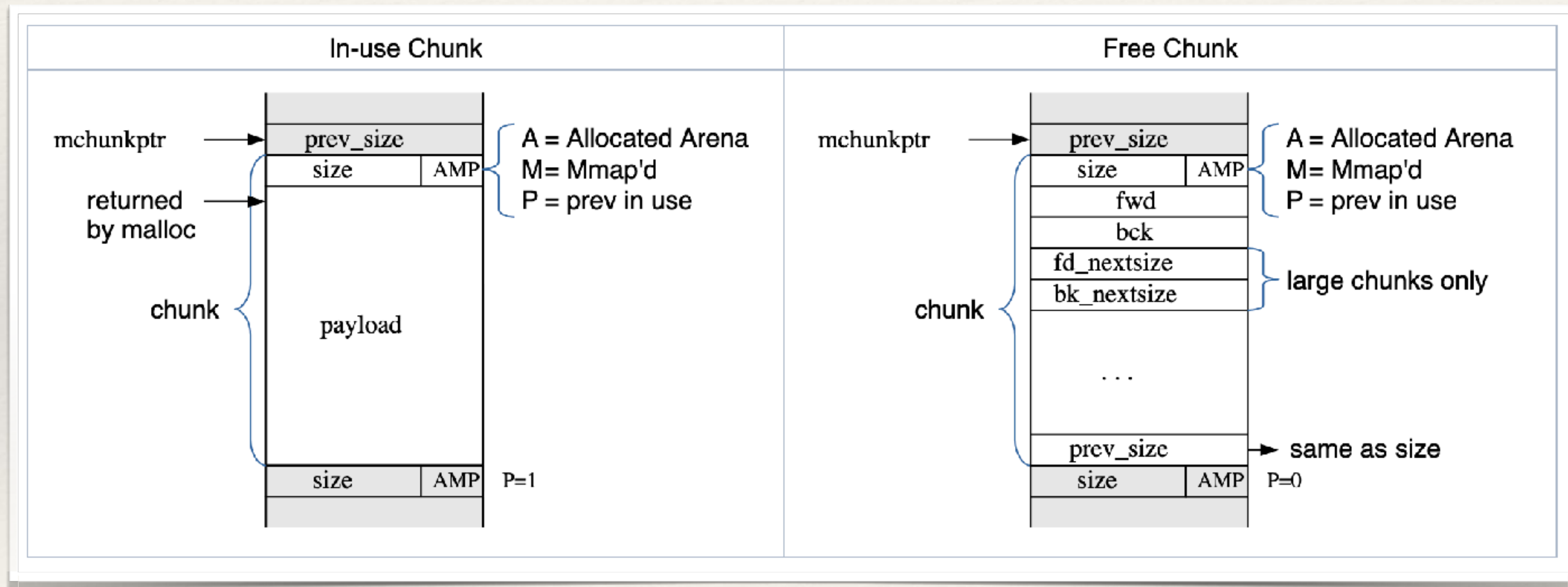
```
16 #include <stdio.h>
15 #include <stdlib.h>
14 #include <string.h>
13
12 char *copy_string(const char *src) {
11     size_t len = strlen(src);
10
9     char *dst = malloc(len);
8     if (dst == NULL)
7         return NULL;
6
5     strcpy(dst, src);
4
3     return dst;
2 }
1
17 int main() {
1     char *s = copy_string("hello");
2
3     printf("%s\n", s);
4
5     free(s);
6     return 0;
7 }

> hse valgrind --leak-check=full ./memory
==1549918== Memcheck, a memory error detector
==1549918== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==1549918== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==1549918== Command: ./memory
==1549918==
==1549918== Invalid write of size 1
==1549918==   at 0x484EE8E: strcpy (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==1549918==   by 0x109219: copy_string (memory.c:12)
==1549918==   by 0x10923A: main (memory.c:18)
==1549918== Address 0x4a91045 is 0 bytes after a block of size 5 alloc'd
==1549918==   at 0x4848899: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==1549918==   by 0x1091F4: copy_string (memory.c:8)
==1549918==   by 0x10923A: main (memory.c:18)
==1549918==
==1549918== Invalid read of size 1
==1549918==   at 0x484ED24: strlen (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==1549918==   by 0x48E5E67: puts (ioputs.c:35)
==1549918==   by 0x10924A: main (memory.c:20)
==1549918== Address 0x4a91045 is 0 bytes after a block of size 5 alloc'd
==1549918==   at 0x4848899: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==1549918==   by 0x1091F4: copy_string (memory.c:8)
==1549918==   by 0x10923A: main (memory.c:18)
==1549918==
hello
==1549918==
==1549918== HEAP SUMMARY:
==1549918==   in use at exit: 0 bytes in 0 blocks
==1549918==   total heap usage: 2 allocs, 2 frees, 1,029 bytes allocated
==1549918==
==1549918== All heap blocks were freed — no leaks are possible
==1549918==
==1549918== For lists of detected and suppressed errors, rerun with: -s
==1549918== ERROR SUMMARY: 2 errors from 2 contexts (suppressed: 0 from 0)
> hse
```

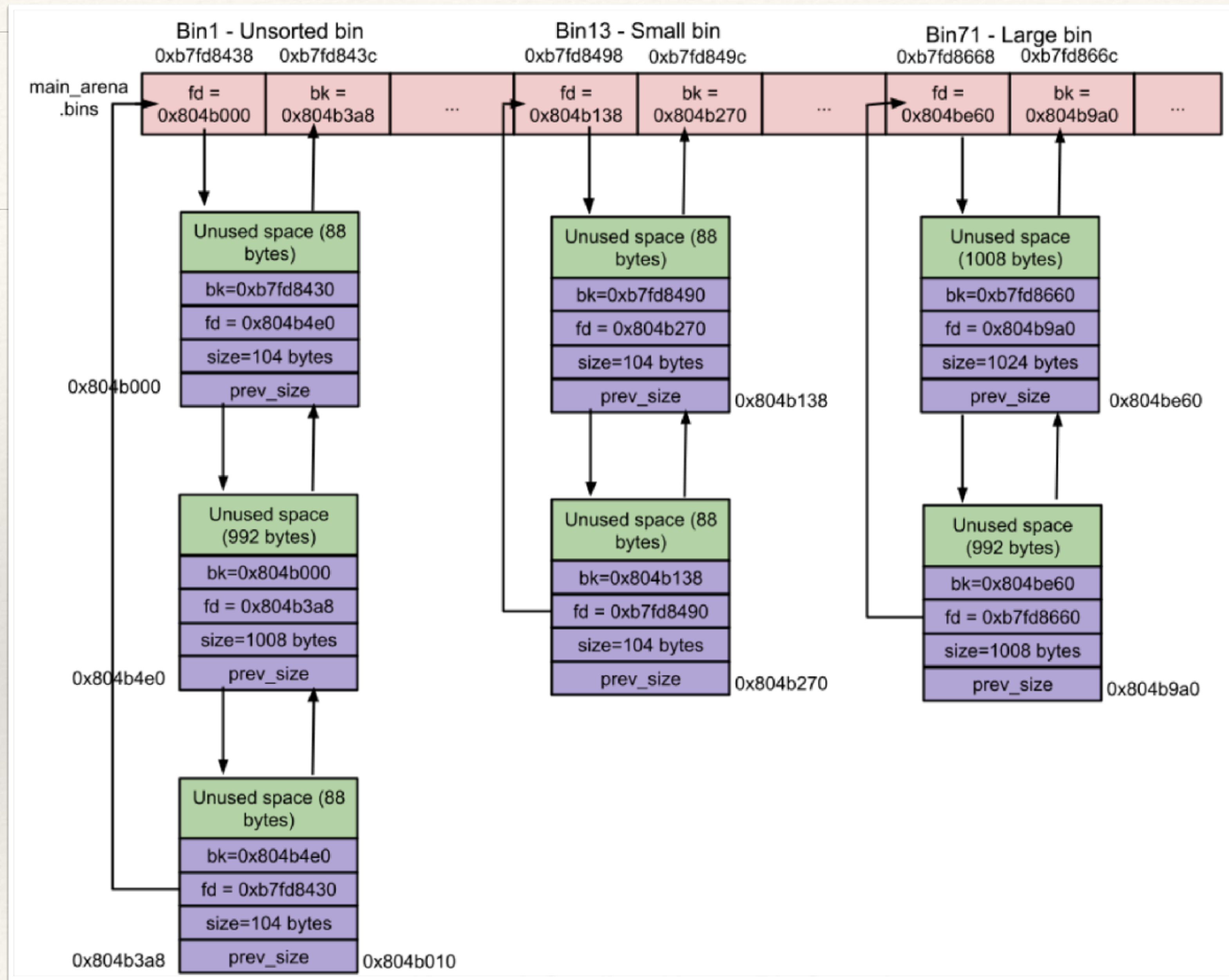
Аллокатор



Блок памяти



<https://sourceware.org/glibc/wiki/MallocInternals>



Размер чанка

```
5 void print_chunk_size(size_t size) {
6     void *p = malloc(size);
7     if (p == NULL) {
8         printf("Can't allocate %zu\n", size);
9         printf("-----\n");
10        return;
11    }
12
13    size_t raw = ((size_t *)p)[-1];
14
15    printf("requested size : %zu\n", size);
16    printf("chunk size      : %zu\n", raw & ~(size_t)0x7);
17    printf("-----\n");
18
19    free(p);
20 }
21
22 int main() {
23     print_chunk_size(6);
24     print_chunk_size(99);
25     print_chunk_size(999);
26     print_chunk_size(0);
27 }
```

x86-64 gcc 16.2

Program returned: 0

Program stdout

requested size : 6
chunk size : 32

requested size : 99
chunk size : 112

requested size : 999
chunk size : 1008

requested size : 0
chunk size : 32

Аллокатор старается переиспользовать блоки

```
5  int main() {  
6      void *p = malloc(4096);  
7  
8      printf("first malloc:  %p\n", p);  
9  
10     free(p);  
11  
12     void *q = malloc(64);  
13  
14     printf("second malloc: %p\n", q);  
15  
16     free(q);  
17 }
```

x86-64 gcc 16.2



Program returned: 0

Program stdout

first malloc: 0x2a7122a0

second malloc: 0x2a7122a0

<https://godbolt.org/z/GndaobYrY>

Overcommit

```
10 #include <stdio.h>
9  #include <stdlib.h>
8
7  int main() {
6      size_t size = 3000ULL * 1024 * 1024 * 1024;
5      void* ptr = malloc(size);
4      if (ptr == NULL) {
3          perror("malloc");
2          return 1;
1      }
11  getchar();
1  for (size_t i = 0; i < size; i += 4096)
2      ((char *)ptr)[i] = 1;
3
4      free(ptr);
5      return 0;
6  }
```

```
> hse ./a.out
^C
> hse gcc malloc.c
> hse ./a.out
```

0[|||||] 50.7% 3[|||||] 26.0% 6[||||] 18.0%
1[|||||] 46.7% 4[|||||] 20.0% 7[||||] 6.7%
2[|||||] 37.6% 5[|||||] 13.9% 8[||||] 2.0%
Mem[|||||] 11.7G/36.0G Tasks: 679, 4214 thr, 0
Swp[|||||] 8.56G/9.00G Load average: 3.27 3.39
Uptime: 6 days, 11:55:07

Main										
PID	USER	PRI	NI	VIRT	RES	S	CPU%	MEM%	TIME+	Command
94338	mfilitov	25	0	3391G	736	?	0.0	0.0	0:00.00	./a.out

OOM-killer

```
10 #include <stdio.h>
9  #include <stdlib.h>
8
7  int main() {
6      size_t size = 3000ULL * 1024 * 1024 * 1024;
5      void* ptr = malloc(size);
4      if (ptr == NULL) {
3          perror("malloc");
2          return 1;
1      }
1      getchar();
1      for (size_t i = 0; i < size; i += 4096)
2          ((char *)ptr)[i] = 1;
3
4      free(ptr);
5      return 0;
6  }
```

→ hse ./a.out

^C

→ hse gcc malloc.c

→ hse ./a.out

[1] 94338 killed ./a.out

→ hse

→ hse

Следующее занятие

- ❖ asm x86-64: регистры, арифметика, переходы